

```
#!/bin/bash

# =====
# FILE: scripts/setup.sh
# Initial setup script
# =====

echo "Kafka Lab Setup Script"
echo "======"
echo ""

# Check Docker
if ! command -v docker &> /dev/null; then
    echo "✗ Docker is not installed"
    exit 1
fi
echo "✓ Docker found"

# Check Docker Compose
if ! command -v docker-compose &> /dev/null; then
    echo "✗ Docker Compose is not installed"
    exit 1
fi
echo "✓ Docker Compose found"

# Check Go
if ! command -v go &> /dev/null; then
    echo "✗ Go is not installed"
    exit 1
fi
echo "✓ Go found ($(go version))"

echo ""
echo "Choose setup:"
echo "1) Single broker (quick start)"
echo "2) 3-broker cluster (production-like)"
read -p "Enter choice [1-2]: " choice

if [ "$choice" = "1" ]; then
    echo ""
    echo "Starting single broker setup..."
    docker-compose -f docker-compose-single.yml up -d
```

```

echo "Waiting for Kafka to be ready..."
sleep 30

echo ""
echo "✓ Single broker Kafka is ready!"
echo ""
echo "Services:"
echo " - Kafka: localhost:9092"
echo " - Zookeeper: localhost:2181"
echo " - Kafka UI: http://localhost:8080"

elif [ "$choice" = "2" ]; then
    echo ""
    echo "Starting 3-broker cluster..."
    docker-compose -f docker-compose-cluster.yml up -d

    echo "Waiting for cluster to be ready..."
    sleep 40

    echo ""
    echo "✓ Kafka cluster is ready!"
    echo ""
    echo "Services:"
    echo " - Kafka Broker 1: localhost:9092"
    echo " - Kafka Broker 2: localhost:9093"
    echo " - Kafka Broker 3: localhost:9094"
    echo " - Zookeeper: localhost:2181"
    echo " - Kafka UI: http://localhost:8080"
else
    echo "Invalid choice"
    exit 1
fi

echo ""
echo "Next steps:"
echo " 1. Open Kafka UI: http://localhost:8080"
echo " 2. Run: ./scripts/create-topics.sh"
echo " 3. Run examples: cd examples && go run simple-producer.go"

---

# =====
# FILE: scripts/create-topics.sh
# Create example topics

```

```

# =====

#!/bin/bash

echo "Creating Kafka Topics"
echo "======"
echo ""

BROKER="localhost:9092"

# Check if Kafka is running
if ! docker ps | grep -q kafka; then
    echo "✗ Kafka is not running. Start it with:"
    echo "  docker-compose -f docker-compose-single.yml up -d"
    exit 1
fi

# Function to create topic
create_topic() {
    local topic=$1
    local partitions=$2
    local replication=$3

    echo "Creating topic: $topic (partitions=$partitions, replication=$replication)"

    docker exec -it kafka kafka-topics \
        --bootstrap-server $BROKER \
        --create \
        --topic $topic \
        --partitions $partitions \
        --replication-factor $replication \
        --if-not-exists 2>/dev/null

    if [ $? -eq 0 ]; then
        echo "✓ Topic $topic created"
    else
        echo "✗ Failed to create $topic"
    fi
}

# Create topics
create_topic "my-first-topic" 3 1
create_topic "go-topic" 3 1
create_topic "orders" 6 1

```

```

create_topic "orders-dlq" 1 1
create_topic "payments" 3 1
create_topic "events" 6 1
create_topic "perf-test" 6 1

echo ""
echo "Listing all topics:"
docker exec -it kafka kafka-topics --bootstrap-server $BROKER --list

echo ""
echo "✓ Topics created successfully!"

---

# =====
# FILE: scripts/test-producer.sh
# Test producer with sample data
# =====

#!/bin/bash

echo "Kafka Test Producer"
echo "=====
echo ""

BROKER="localhost:9092"
TOPIC="{1:-my-first-topic}"
COUNT="{2:-10}"

echo "Topic: $TOPIC"
echo "Messages: $COUNT"
echo ""

# Generate and send messages
for i in $(seq 1 $COUNT); do
    timestamp=$(date +%s)
    message="{\"id\":$i,\"message\":\"Test message $i\",\"timestamp\":$timestamp}"

    echo $message | docker exec -i kafka kafka-console-producer \
        --bootstrap-server $BROKER \
        --topic $TOPIC 2>/dev/null

    echo "Sent: $message"
    sleep 0.5

```

done

echo ""

echo "✓ Sent \$COUNT messages to \$TOPIC"

=====

FILE: scripts/test-consumer.sh

Test consumer

=====

#!/bin/bash

echo "Kafka Test Consumer"

echo "=====

echo ""

BROKER="localhost:9092"

TOPIC="\${1:-my-first-topic}"

GROUP="\${2:-test-group}"

echo "Topic: \$TOPIC"

echo "Group: \$GROUP"

echo ""

echo "Press Ctrl+C to stop"

echo "---"

docker exec -it kafka kafka-console-consumer \

--bootstrap-server \$BROKER \

--topic \$TOPIC \

--group \$GROUP \

--from-beginning \

--property print.timestamp=true \

--property print.key=true \

--property print.value=true

=====

FILE: scripts/check-lag.sh

Monitor consumer lag

=====

```
#!/bin/bash
```

```
BROKER="localhost:9092"
```

```
GROUP="${1}"
```

```
if [ -z "$GROUP" ]; then
```

```
    echo "Usage: ./check-lag.sh <consumer-group>"
```

```
    echo ""
```

```
    echo "Available groups:"
```

```
    docker exec -it kafka kafka-consumer-groups --bootstrap-server $BROKER --list
```

```
    exit 1
```

```
fi
```

```
echo "Monitoring Consumer Lag for: $GROUP"
```

```
echo "=====
```

```
echo ""
```

```
echo "Press Ctrl+C to stop"
```

```
echo ""
```

```
watch -n 2 "docker exec kafka kafka-consumer-groups \
```

```
    --bootstrap-server $BROKER \
```

```
    --group $GROUP \
```

```
    --describe"
```

```
---
```

```
# =====
```

```
# FILE: scripts/reset-offsets.sh
```

```
# Reset consumer group offsets
```

```
# =====
```

```
#!/bin/bash
```

```
BROKER="localhost:9092"
```

```
GROUP="${1}"
```

```
TOPIC="${2}"
```

```
RESET_TO="${3:-earliest}"
```

```
if [ -z "$GROUP" ] || [ -z "$TOPIC" ]; then
```

```
    echo "Usage: ./reset-offsets.sh <consumer-group> <topic> [earliest|latest|offset]"
```

```
    echo ""
```

```
    echo "Examples:"
```

```
    echo "  ./reset-offsets.sh my-group my-topic earliest"
```

```
    echo "  ./reset-offsets.sh my-group my-topic latest"
```

```
    exit 1
fi

echo "Resetting offsets for group: $GROUP, topic: $TOPIC"
echo "Reset to: $RESET_TO"
echo ""

# Show current offsets
echo "Current offsets:"
docker exec kafka kafka-consumer-groups \
    --bootstrap-server $BROKER \
    --group $GROUP \
    --describe

echo ""
read -p "Continue with reset? [y/N] " confirm

if [ "$confirm" != "y" ]; then
    echo "Cancelled"
    exit 0
fi

# Reset offsets
docker exec kafka kafka-consumer-groups \
    --bootstrap-server $BROKER \
    --group $GROUP \
    --topic $TOPIC \
    --reset-offsets \
    --to-$RESET_TO \
    --execute

echo ""
echo "✓ Offsets reset to $RESET_TO"

# Show new offsets
echo ""
echo "New offsets:"
docker exec kafka kafka-consumer-groups \
    --bootstrap-server $BROKER \
    --group $GROUP \
    --describe

---
```

```

# =====
# FILE: scripts/topic-info.sh
# Get detailed topic information
# =====

#!/bin/bash

BROKER="localhost:9092"
TOPIC="${1}"

if [ -z "$TOPIC" ]; then
    echo "Usage: ./topic-info.sh <topic-name>"
    echo ""
    echo "Available topics:"
    docker exec kafka kafka-topics --bootstrap-server $BROKER --list
    exit 1
fi

echo "Topic Information: $TOPIC"
echo "=====
echo ""

# Describe topic
echo "Configuration:"
docker exec kafka kafka-topics \
    --bootstrap-server $BROKER \
    --describe \
    --topic $TOPIC

echo ""
echo "Messages count per partition:"
docker exec kafka kafka-run-class kafka.tools.GetOffsetShell \
    --broker-list $BROKER \
    --topic $TOPIC \
    --time -1

echo ""
echo "Topic configs:"
docker exec kafka kafka-configs \
    --bootstrap-server $BROKER \
    --entity-type topics \
    --entity-name $TOPIC \
    --describe

```

```
# =====  
# FILE: scripts/produce-json.sh  
# Produce JSON messages  
# =====
```

```
#!/bin/bash
```

```
BROKER="localhost:9092"
```

```
TOPIC="${1:-orders}"
```

```
COUNT="${2:-20}"
```

```
echo "Producing JSON messages to: $TOPIC"
```

```
echo "=====
```

```
echo ""
```

```
for i in $(seq 1 $COUNT); do
```

```
    order_id="ORD-$(printf "%05d" $i)"
```

```
    amount=$(awk -v min=10 -v max=500 'BEGIN{srand(); print min+rand()*(max-min)}')
```

```
    timestamp=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
```

```
    json="{\"orderId\": \"$order_id\", \"amount\": $amount, \"timestamp\": \"$timestamp\"}"
```

```
    echo $json | docker exec -i kafka kafka-console-producer \
```

```
        --bootstrap-server $BROKER \
```

```
        --topic $TOPIC \
```

```
        --property "parse.key=true" \
```

```
        --property "key.separator=: " <<< "$order_id:$json" 2>/dev/null
```

```
    echo "✓ Produced: $json"
```

```
    sleep 0.2
```

```
done
```

```
echo ""
```

```
echo "✓ Produced $COUNT JSON messages to $TOPIC"
```

```
# =====  
# FILE: scripts/consume-json.sh  
# Consume and format JSON messages  
# =====
```

```
#!/bin/bash
```

```
BROKER="localhost:9092"
```

```
TOPIC="${1:-orders}"
```

```
GROUP="${2:-json-consumer}"
```

```
echo "Consuming JSON from: $TOPIC"
```

```
echo "=====
```

```
echo ""
```

```
docker exec -it kafka kafka-console-consumer \
```

```
--bootstrap-server $BROKER \
```

```
--topic $TOPIC \
```

```
--group $GROUP \
```

```
--from-beginning \
```

```
--property print.timestamp=true \
```

```
--property print.key=true \
```

```
--property print.value=true \
```

```
--formatter kafka.tools.DefaultMessageFormatter
```

```
---
```

```
# =====
```

```
# FILE: scripts/performance-test.sh
```

```
# Run performance tests
```

```
# =====
```

```
#!/bin/bash
```

```
BROKER="localhost:9092"
```

```
TOPIC="perf-test"
```

```
NUM_RECORDS=100000
```

```
RECORD_SIZE=1024
```

```
THROUGHPUT=-1
```

```
echo "Kafka Performance Test"
```

```
echo "=====
```

```
echo ""
```

```
echo "Topic: $TOPIC"
```

```
echo "Records: $NUM_RECORDS"
```

```
echo "Record Size: $RECORD_SIZE bytes"
```

```
echo "Target Throughput: unlimited"
```

```
echo ""
```

```

# Create topic if not exists
docker exec kafka kafka-topics \
  --bootstrap-server $BROKER \
  --create \
  --topic $TOPIC \
  --partitions 6 \
  --replication-factor 1 \
  --if-not-exists 2>/dev/null

echo "Running producer performance test..."
echo "---"

docker exec kafka kafka-producer-perf-test \
  --topic $TOPIC \
  --num-records $NUM_RECORDS \
  --record-size $RECORD_SIZE \
  --throughput $THROUGHPUT \
  --producer-props bootstrap.servers=$BROKER \
  acks=1 \
  compression.type=lz4

echo ""
echo "---"
echo "Running consumer performance test..."
echo "---"

docker exec kafka kafka-consumer-perf-test \
  --bootstrap-server $BROKER \
  --topic $TOPIC \
  --messages $NUM_RECORDS \
  --threads 4 \
  --group perf-test-group

echo ""
echo "✓ Performance test completed"

---

# =====
# FILE: scripts/cleanup.sh
# Cleanup all Kafka data and containers
# =====

#!/bin/bash

```

```

echo "Kafka Cleanup Script"
echo "======"
echo ""
echo "This will:"
echo " - Stop all Kafka containers"
echo " - Remove all volumes (DELETE ALL DATA)"
echo " - Remove all Kafka topics"
echo ""
read -p "Are you sure? [y/N] " confirm

if [ "$confirm" != "y" ]; then
    echo "Cancelled"
    exit 0
fi

echo ""
echo "Stopping containers..."
docker-compose -f docker-compose-single.yml down -v 2>/dev/null
docker-compose -f docker-compose-cluster.yml down -v 2>/dev/null

echo "Removing volumes..."
docker volume rm $(docker volume ls -q | grep kafka) 2>/dev/null
docker volume rm $(docker volume ls -q | grep zookeeper) 2>/dev/null

echo "Cleaning up networks..."
docker network prune -f

echo ""
echo "✓ Cleanup completed"
echo ""
echo "To start again, run: ./scripts/setup.sh"

---

# =====
# FILE: scripts/broker-health.sh
# Check broker health
# =====

#!/bin/bash

BROKER="localhost:9092"

```

```

echo "Kafka Broker Health Check"
echo "=====
echo ""

# Check if containers are running
echo "Container Status:"
echo "---"
docker ps --filter "name=kafka" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
echo ""

# Check broker API versions
echo "Broker API Versions:"
echo "---"
docker exec kafka kafka-broker-api-versions --bootstrap-server $BROKER 2>/dev/null | head -5
echo ""

# Check under-replicated partitions
echo "Under-Replicated Partitions:"
echo "---"
result=$(docker exec kafka kafka-topics --bootstrap-server $BROKER --describe --under-
replicated-partitions 2>/dev/null)
if [ -z "$result" ]; then
    echo "✓ No under-replicated partitions"
else
    echo "$result"
fi
echo ""

# Check unavailable partitions
echo "Unavailable Partitions:"
echo "---"
result=$(docker exec kafka kafka-topics --bootstrap-server $BROKER --describe --unavailable-
partitions 2>/dev/null)
if [ -z "$result" ]; then
    echo "✓ No unavailable partitions"
else
    echo "$result"
fi
echo ""

# Disk usage
echo "Disk Usage:"
echo "---"
docker exec kafka du -sh /var/lib/kafka/data 2>/dev/null

```

```

echo ""

# Active consumer groups
echo "Active Consumer Groups:"
echo "---"
docker exec kafka kafka-consumer-groups --bootstrap-server $BROKER --list 2>/dev/null
echo ""

echo "✓ Health check completed"

---

# =====
# FILE: scripts/simulate-failure.sh
# Simulate broker failure (for cluster only)
# =====

#!/bin/bash

echo "Kafka Failure Simulation"
echo "======"
echo ""

if ! docker ps | grep -q "kafka-2"; then
    echo "✗ This script requires the 3-broker cluster"
    echo "Start it with: docker-compose -f docker-compose-cluster.yml up -d"
    exit 1
fi

echo "This will simulate a broker failure by stopping kafka-2"
echo ""
read -p "Continue? [y/N] " confirm

if [ "$confirm" != "y" ]; then
    echo "Cancelled"
    exit 0
fi

echo ""
echo "Stopping kafka-2..."
docker stop kafka-2

echo "Waiting 5 seconds..."
sleep 5

```

```
echo ""
echo "Cluster status:"
docker ps --filter "name=kafka" --format "table {{.Names}}\t{{.Status}}"
```

```
echo ""
echo "Checking partition leadership:"
docker exec kafka-1 kafka-topics \
  --bootstrap-server localhost:9092 \
  --describe 2>/dev/null | grep "Leader: 2"
```

```
echo ""
echo "Under-replicated partitions:"
docker exec kafka-1 kafka-topics \
  --bootstrap-server localhost:9092 \
  --describe \
  --under-replicated-partitions
```

```
echo ""
echo "To restore kafka-2, run: docker start kafka-2"
```

```
# =====
# FILE: scripts/restore-broker.sh
# Restore failed broker
# =====
```

```
#!/bin/bash
```

```
echo "Restoring Kafka Broker"
echo "===== "
echo ""
```

```
BROKER="${1:-kafka-2}"
```

```
echo "Restarting $BROKER..."
docker start $BROKER
```

```
echo "Waiting for broker to rejoin cluster..."
sleep 10
```

```
echo ""
echo "Cluster status:"
```

```
docker ps --filter "name=kafka" --format "table {{.Names}}\t{{.Status}}"
```

```
echo ""
```

```
echo "Checking replication:"
```

```
docker exec kafka-1 kafka-topics \
  --bootstrap-server localhost:9092 \
  --describe \
  --under-replicated-partitions 2>/dev/null
```

```
if [ $? -eq 0 ]; then
```

```
  echo ""
```

```
  echo "✓ Broker restored successfully"
```

```
else
```

```
  echo ""
```

```
  echo "⚠ Check logs: docker logs $BROKER"
```

```
fi
```

```
---
```

```
# =====
```

```
# FILE: scripts/logs.sh
```

```
# View Kafka logs
```

```
# =====
```

```
#!/bin/bash
```

```
CONTAINER="${1:-kafka}"
```

```
LINES="${2:-100}"
```

```
echo "Kafka Logs: $CONTAINER (last $LINES lines)"
```

```
echo "=====
```

```
echo ""
```

```
if ! docker ps --format "{{.Names}}" | grep -q "^$CONTAINER$"; then
```

```
  echo "✗ Container $CONTAINER not found"
```

```
  echo ""
```

```
  echo "Available containers:"
```

```
  docker ps --filter "name=kafka" --format "{{.Names}}"
```

```
  exit 1
```

```
fi
```

```
# Show logs
```

```
docker logs $CONTAINER --tail $LINES --follow
```

```
# =====  
# FILE: scripts/quick-test.sh  
# Quick end-to-end test  
# =====
```

```
#!/bin/bash
```

```
BROKER="localhost:9092"  
TOPIC="quick-test"
```

```
echo "Kafka Quick Test"  
echo "=====  
echo ""
```

```
# Create topic  
echo "1. Creating topic..."  
docker exec kafka kafka-topics \  
  --bootstrap-server $BROKER \  
  --create \  
  --topic $TOPIC \  
  --partitions 3 \  
  --replication-factor 1 \  
  --if-not-exists 2>/dev/null
```

```
# Produce messages  
echo "2. Producing 5 messages..."  
for i in {1..5}; do  
  echo "Test message $i" | docker exec -i kafka kafka-console-producer \  
    --bootstrap-server $BROKER \  
    --topic $TOPIC 2>/dev/null  
done
```

```
# Consume messages  
echo "3. Consuming messages..."  
docker exec kafka kafka-console-consumer \  
  --bootstrap-server $BROKER \  
  --topic $TOPIC \  
  --from-beginning \  
  --max-messages 5 \  
  --timeout-ms 5000 2>/dev/null
```

```
echo ""
```

```
echo "✓ Quick test completed successfully!"
```

```
# Cleanup
echo ""
read -p "Delete test topic? [y/N] " cleanup
if [ "$cleanup" = "y" ]; then
    docker exec kafka kafka-topics \
        --bootstrap-server $BROKER \
        --delete \
        --topic $TOPIC 2>/dev/null
    echo "✓ Test topic deleted"
fi
```

```
---
```

```
# =====
# FILE: Makefile
# Convenient make commands
# =====
```

```
.PHONY: help setup start-single start-cluster stop clean
```

```
help:
```

```
@echo "Kafka Lab Makefile"
@echo "===== "
@echo ""
@echo "Commands:"
@echo " make setup      - Run initial setup"
@echo " make start-single - Start single broker"
@echo " make start-cluster - Start 3-broker cluster"
@echo " make stop        - Stop all containers"
@echo " make clean       - Stop and remove all data"
@echo " make test        - Run quick test"
@echo " make topics      - Create example topics"
@echo " make health      - Check broker health"
@echo " make logs        - View Kafka logs"
```

```
setup:
```

```
@chmod +x scripts/*.sh
@./scripts/setup.sh
```

```
start-single:
```

```
@docker-compose -f docker-compose-single.yml up -d
@echo "Waiting for Kafka..."
```

```
@sleep 30
@echo "✓ Kafka ready at localhost:9092"
@echo "✓ Kafka UI at http://localhost:8080"
```

start-cluster:

```
@docker-compose -f docker-compose-cluster.yml up -d
@echo "Waiting for cluster..."
@sleep 40
@echo "✓ Kafka cluster ready"
@echo " Broker 1: localhost:9092"
@echo " Broker 2: localhost:9093"
@echo " Broker 3: localhost:9094"
@echo "✓ Kafka UI at http://localhost:8080"
```

stop:

```
@docker-compose -f docker-compose-single.yml down 2>/dev/null || true
@docker-compose -f docker-compose-cluster.yml down 2>/dev/null || true
@echo "✓ Kafka stopped"
```

clean:

```
@./scripts/cleanup.sh
```

test:

```
@./scripts/quick-test.sh
```

topics:

```
@./scripts/create-topics.sh
```

health:

```
@./scripts/broker-health.sh
```

logs:

```
@./scripts/logs.sh kafka 100
```

```
# =====
# FILE: README-SCRIPTS.md
# Scripts documentation
# =====
```

```
# Kafka Lab Scripts Documentation
```

Setup Scripts

setup.sh

Initial setup - Choose between single broker or cluster

```
```bash
```

```
./scripts/setup.sh
```

```
```
```

create-topics.sh

Create all example topics

```
```bash
```

```
./scripts/create-topics.sh
```

```
```
```

Testing Scripts

quick-test.sh

Quick end-to-end test

```
```bash
```

```
./scripts/quick-test.sh
```

```
```
```

test-producer.sh

Send test messages to a topic

```
```bash
```

```
./scripts/test-producer.sh <topic> <count>
```

# Example:

```
./scripts/test-producer.sh my-topic 100
```

```
```
```

test-consumer.sh

Consume messages from a topic

```
```bash
```

```
./scripts/test-consumer.sh <topic> <group>
```

# Example:

```
./scripts/test-consumer.sh my-topic test-group
```

```
```
```

produce-json.sh

Produce JSON formatted messages

```
```bash
```

```
./scripts/produce-json.sh <topic> <count>
```

# Example:

```
./scripts/produce-json.sh orders 50
```

```
...
```

```
consume-json.sh
Consume and display JSON messages
```bash
./scripts/consume-json.sh <topic> <group>
```
```

```
performance-test.sh
Run producer and consumer performance tests
```bash
./scripts/performance-test.sh
```
```

## ## Monitoring Scripts

```
check-lag.sh
Monitor consumer group lag in real-time
```bash
./scripts/check-lag.sh <consumer-group>
```
```

```
topic-info.sh
Get detailed topic information
```bash
./scripts/topic-info.sh <topic>
```
```

```
broker-health.sh
Check broker health status
```bash
./scripts/broker-health.sh
```
```

```
logs.sh
View Kafka logs
```bash
./scripts/logs.sh <container> <lines>
# Example:
./scripts/logs.sh kafka 200
./scripts/logs.sh kafka-1 100
```
```

## ## Management Scripts

```
reset-offsets.sh
Reset consumer group offsets
```bash
./scripts/reset-offsets.sh <group> <topic> [earliest|latest]
# Examples:
./scripts/reset-offsets.sh my-group my-topic earliest
./scripts/reset-offsets.sh my-group my-topic latest
```
```

```
simulate-failure.sh
Simulate broker failure (cluster only)
```bash
./scripts/simulate-failure.sh
```
```

```
restore-broker.sh
Restore a failed broker
```bash
./scripts/restore-broker.sh <broker-name>
# Example:
./scripts/restore-broker.sh kafka-2
```
```

```
cleanup.sh
Stop all containers and delete all data
```bash
./scripts/cleanup.sh
```
```

## ## Makefile Commands

```
```bash
make help      # Show all commands
make setup     # Run initial setup
make start-single # Start single broker
make start-cluster # Start 3-broker cluster
make stop      # Stop all containers
make clean     # Stop and remove all data
make test      # Run quick test
make topics    # Create example topics
make health    # Check broker health
make logs      # View Kafka logs
```
```

## ## Usage Examples

### ### Example 1: Full Lab Setup

```
```bash
```

```
# Initial setup  
make setup
```

```
# Create topics  
make topics
```

```
# Run quick test  
make test
```

```
# Check health  
make health  
```
```

### ### Example 2: Testing Producer/Consumer

```
```bash
```

```
# Produce 100 messages  
./scripts/test-producer.sh my-topic 100
```

```
# Consume in one terminal  
./scripts/test-consumer.sh my-topic group-1
```

```
# Monitor lag in another terminal  
./scripts/check-lag.sh group-1  
```
```

### ### Example 3: JSON Messages

```
```bash
```

```
# Produce JSON orders  
./scripts/produce-json.sh orders 50
```

```
# Consume and display  
./scripts/consume-json.sh orders order-processor  
```
```

### ### Example 4: Performance Testing

```
```bash
```

```
# Run performance test  
./scripts/performance-test.sh
```

```
# Check topic info
./scripts/topic-info.sh perf-test
```
```

#### ### Example 5: Failure Simulation

```
```bash
```

```
# Start cluster
make start-cluster
```

```
# Create replicated topic
```

```
docker exec kafka-1 kafka-topics --bootstrap-server localhost:9092 \
  --create --topic fault-test --partitions 6 --replication-factor 3
```

```
# Simulate failure
```

```
./scripts/simulate-failure.sh
```

```
# Check health
```

```
make health
```

```
# Restore broker
```

```
./scripts/restore-broker.sh kafka-2
```
```

#### ## Making Scripts Executable

```
```bash
```

```
chmod +x scripts/*.sh
```

```
```
```

#### ## Troubleshooting

If scripts fail:

1. Check Docker is running: `docker ps`
2. Check Kafka is ready: `make health`
3. View logs: `make logs`
4. Restart: `make stop && make start-single`